

Linear regression

Linear regression is a technique that is used to model the relationship between a **dependent variable** (Response variable) and one or more **independent variables** (predictors).

Here's a breakdown of its key aspects:

1. The Goal: The primary goal of linear regression is to find a "best-fit" straight line (or hyperplane in higher dimensions) that describes the linear relationship between these variables. This line allows you to:

- **Predict** the value of the dependent variable for new, unseen values of the independent variables.
- **Understand** the strength and direction of the relationship between variables. For example, does increasing advertising spending (independent variable) lead to an increase or decrease in sales (dependent variable), and by how much?

2. The Linear Equation: The core of linear regression is a linear equation.

- **Simple Linear Regression (one independent variable):** $Y = \beta_0 + \beta_1 X + \epsilon$

Where:

- Y is the dependent variable (what you're predicting).
- X is the independent variable (the predictor).
- β_0 is the **y-intercept** (the predicted value of Y when X is 0).
- β_1 is the **slope** (how much Y is expected to change for a one-unit increase in X).
- ϵ is the **error term** or residual, representing the difference between the actual and predicted values.

- **Multiple Linear Regression (two or more independent variables):**

$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n + \epsilon$ Here, you have multiple independent variables ($X_1, X_2, \dots, X_n$), each with its own slope coefficient ($\beta_1, \beta_2, \dots, \beta_n$).

3. How it Works (Least Squares Method): Linear regression typically uses the **Ordinary Least Squares (OLS)** method to find the best-fit line. This method works by minimizing the **sum of the squared differences** between the actual observed data points and the values predicted by the regression line. These differences are called **residuals**. By minimizing these squared residuals, the line is positioned to be as close as possible to all the data points.

4. Key Assumptions: For the results of linear regression to be reliable and interpretable, several assumptions about the data should ideally hold true:

- **Linearity:** The relationship between the independent and dependent variables should be linear.

- **Independence of Errors:** The errors (residuals) should be independent of each other (no patterns or correlations).
- **Homoscedasticity:** The variance of the errors should be constant across all values of the independent variable.
- **Normality of Residuals:** The residuals should ideally follow a normal distribution.

5. Applications: Linear regression is widely used in various fields for:

- **Forecasting and Prediction:** Predicting sales, stock prices, housing prices, etc.
- **Understanding Relationships:** Determining the impact of marketing spend on revenue, the relationship between education level and income, etc.
- **Trend Analysis:** Identifying trends in data over time.

Polynomial regression

Polynomial regression is an extension of linear regression that allows you to model non-linear relationships between variables. While linear regression fits a straight line to the data, polynomial regression fits a curved line.

Here's a deeper dive:

1. The Need for Polynomial Regression: Linear regression works well when the relationship between the independent variable(s) and the dependent variable is truly linear. However, in many real-world scenarios, this isn't the case. Data can exhibit curves, bends, or other non-linear patterns. If you try to fit a straight line to such data, the model will likely perform poorly, leading to inaccurate predictions and a poor understanding of the underlying relationship. This is where polynomial regression comes in handy.

2. The Polynomial Equation: Instead of a simple linear equation ($Y = \beta_0 + \beta_1 X + \epsilon$), polynomial regression uses a polynomial equation of a certain degree. For a single independent variable X , the equation for a polynomial regression of degree n is:

$$Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \beta_3 X^3 + \dots + \beta_n X^n + \epsilon$$

Where:

- Y is the dependent variable.
- X is the independent variable.
- $\beta_0, \beta_1, \dots, \beta_n$ are the coefficients (parameters) that the model learns.
- n is the **degree** of the polynomial. This determines the flexibility of the curve.
 - If $n = 1$, it's a simple linear regression.
 - If $n = 2$, it's a quadratic regression (parabola).
 - If $n = 3$, it's a cubic regression, and so on.
- ϵ is the error term.

3. How it Works (Linear in Parameters): Even though polynomial regression models a non-linear relationship between X and Y , it's still considered a **linear model in terms of its coefficients (β values)**. This is a crucial point.

To understand this, consider a quadratic polynomial regression:

$$Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \epsilon$$

You can think of X^2 as a new, transformed independent variable, say $Z = X^2$. Then the equation becomes: $Y = \beta_0 + \beta_1 X + \beta_2 Z + \epsilon$

This looks exactly like a multiple linear regression equation, where you have multiple independent variables (X and Z). Because the model is linear with respect to the β coefficients, the same techniques used for linear regression (like Ordinary Least Squares to minimize the sum of squared residuals) can be applied to find the best-fit polynomial curve.

4. Choosing the Degree: The choice of the polynomial degree (n) is critical:

- **Low degree:** Might lead to **underfitting**, where the model is too simple to capture the complexity of the data, resulting in high bias.
- **High degree:** Can lead to **overfitting**, where the model fits the training data too closely, including noise, and performs poorly on new, unseen data, resulting in high variance. You typically need to find an optimal degree that balances bias and variance. Techniques like cross-validation can help in this selection.

5. Advantages:

- **Flexibility:** Can model a wide range of non-linear relationships.
- **Improved Fit:** Often provides a better fit to data that exhibits curved patterns compared to simple linear regression.

6. Disadvantages:

- **Interpretability:** Higher-degree polynomials can be harder to interpret, as the individual coefficients don't have as straightforward a meaning as the slope in linear regression.
- **Overfitting Risk:** As mentioned, a high degree significantly increases the risk of overfitting.
- **Extrapolation:** Extrapolating beyond the range of the observed data with a high-degree polynomial can lead to highly unreliable predictions, as the curve can behave erratically outside the data range.

In summary, polynomial regression is a powerful tool when your data clearly shows a non-linear relationship. By transforming the independent variable(s) into polynomial terms, it allows you to fit curves and capture more complex patterns, while still leveraging the computational advantages of linear models.

Mean Squared Error (MSE)

At its core, MSE measures the **average of the squares of the errors**. An "error" in this context is the difference between an actual value (Y_i) and the value predicted by the model ($\hat{Y}_i$).

The Formula:

The formula for Mean Squared Error is:

$$MSE = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$$

Where:

- n is the number of data points (observations).
- Y_i is the actual (observed) value for the i -th data point.
- $\hat{Y}_i$ is the predicted value for the i -th data point.
- $\sum$ denotes the summation across all data points.

How it's Calculated (Step-by-Step):

1. **Calculate the Residuals (Errors):** For each data point, subtract the predicted value from the actual value: $(Y_i - \hat{Y}_i)$.
2. **Square the Errors:** Square each of these differences: $(Y_i - \hat{Y}_i)^2$. Squaring ensures that all errors are positive (whether the prediction was too high or too low, the squared error is positive) and also gives more weight to larger errors.
3. **Sum the Squared Errors:** Add up all the squared errors from step 2. This is often referred to as the Sum of Squared Errors (SSE).
4. **Calculate the Mean:** Divide the sum of squared errors by the total number of data points (n).

Why Square the Errors?

- **Positive Values:** If you just summed the raw errors, positive and negative errors could cancel each other out, potentially giving a misleading impression of a good fit even if there were large errors in both directions. Squaring makes all errors positive.
- **Penalizes Large Errors More Heavily:** Because errors are squared, larger errors contribute disproportionately more to the MSE than smaller errors. For example, an error of 10 squared is 100, while an error of 2 squared is 4. This characteristic makes MSE sensitive to outliers, which can be both an advantage (if you want to heavily penalize models that make big mistakes) and a disadvantage (if outliers are simply noise and not indicative of model performance).

Interpretation of MSE:

- **Lower is Better:** A lower MSE indicates that the model's predictions are closer to the actual values, suggesting a better fit to the data.
- **Zero MSE:** An MSE of zero would mean that the model predicts every single data point perfectly, which is almost never achievable in real-world scenarios due to inherent randomness or unmeasured factors.

- **Scale Dependence:** The magnitude of MSE depends on the scale of the target variable. For example, an MSE of 10 might be considered very high for a model predicting temperatures in Celsius, but very low for a model predicting house prices in thousands of Euros. This makes it difficult to compare MSEs across models predicting different types of values.

Overfitting and Underfitting

Overfitting

What it is: Overfitting occurs when a machine learning model learns the training data *too well*, including the noise and random fluctuations present in that specific dataset, rather than just the underlying patterns. It's like a student who memorises every answer in a textbook without truly understanding the concepts; they might ace the practice test (training data) but fail a new test (unseen data) because they can't apply their knowledge to new problems.

- **Analogy:** Imagine trying to draw a line through a set of scattered data points. An overfit model would draw a highly wiggly line that passes through almost every single point, even the noisy ones. This line might look great for the given points, but it won't accurately predict new points that follow the general trend.

How to prevent overfitting:

- **More Data:** Increase the size and diversity of your training dataset. More data helps the model learn generalized patterns rather than memorizing specific examples.
- **Data Augmentation:** Artificially expand the training dataset by creating modified versions of existing data (e.g., rotating, flipping, or scaling images; adding noise to text).
- **Feature Selection/Engineering:** Reduce the number of input features or create more meaningful features to simplify the model's task. Remove irrelevant or redundant features.
- **Regularization (L1, L2, Dropout):** Add a penalty to the model's loss function to discourage large coefficients or complex models. This encourages simpler models that are less likely to overfit.
 - **L1 Regularization (Lasso):** Adds a penalty proportional to the absolute value of the coefficients. Can lead to sparsity (some coefficients becoming exactly zero), effectively performing feature selection.
 - **L2 Regularization (Ridge):** Adds a penalty proportional to the square of the coefficients. Encourages smaller, more distributed coefficients.
 - **Dropout (for neural networks):** Randomly "drops out" (sets to zero) a percentage of neurons during each training iteration, forcing the network to learn more robust features.
- **Early Stopping:** Monitor the model's performance on a separate validation set during training. Stop training when the performance on the validation set starts

to degrade, even if the training set performance is still improving.

- **Cross-Validation:** Use techniques like K-fold cross-validation to get a more robust estimate of how well the model will generalize to unseen data. This helps in hyperparameter tuning and model selection.
- **Simplify the Model:** Reduce the complexity of your model (e.g., decrease the degree of a polynomial, reduce the number of layers or neurons in a neural network).
- **Ensemble Methods:** Combine predictions from multiple models (e.g., Bagging, Boosting, Random Forests) to reduce variance and improve generalization.

Underfitting

What it is: Underfitting occurs when a machine learning model is too simple to capture the underlying patterns in the training data, leading to poor performance on both the training data and unseen data. It's like a student who doesn't study enough for a test; they don't even understand the basic concepts, so they perform poorly on both practice questions and the actual exam.

- **Analogy:** Using the data point example again, an underfit model would draw a simple straight line through widely scattered data points that clearly have a curved relationship. This line won't fit the existing points well, nor will it predict new points accurately.

How to prevent underfitting:

- **Increase Model Complexity:**
 - Use a more sophisticated model (e.g., switch from linear regression to polynomial regression or a neural network).
 - Add more layers or neurons to a neural network.
 - Increase the number of features or create new, more informative features through **feature engineering** (e.g., creating interaction terms, polynomial features).
- **Decrease Regularization:** If regularization was applied, reduce its strength (e.g., decrease the regularization parameter λ).
- **Train for Longer:** Allow the model to train for more iterations or epochs, provided it's not leading to overfitting.
- **Add More Relevant Features:** If the current features aren't sufficient, find or engineer new features that better explain the target variable.

The Bias-Variance Trade-off

Overfitting and underfitting are often discussed in the context of the **bias-variance trade-off**.

- **Bias:** The error introduced by approximating a real-world problem, which may be complex, by a simplified model. High bias leads to underfitting.

- **Variance:** The amount that the model's prediction would change if it were trained on a different dataset. High variance leads to overfitting.

The goal is to find a model that has a good balance of bias and variance, minimizing the total error on unseen data. This usually means accepting a slight increase in training error to achieve better generalization performance.

The Bias-Variance Decomposition

The **bias-variance decomposition** is a concept that allows us to understand the prediction error of a model. It breaks down the expected error of a model into three distinct components: bias, variance, and irreducible error. This decomposition is crucial for diagnosing model performance and making informed decisions to improve it, as it highlights the inherent tradeoff between a model's simplicity and its flexibility.

Understanding the Components of Error

The primary goal of a supervised machine learning model is to learn a rule that maps input features to an output that is as close as possible to the true underlying relationship. However, in practice, achieving a perfect mapping is impossible due to the inherent randomness and complexity of real-world data. The bias-variance decomposition helps us understand the sources of a model's error.

Let's consider a true, but unknown, function $f(x)$ that we are trying to approximate with our model $\hat{f}(x)$. The expected squared error of our prediction at a point x can be decomposed as follows:

$$E[(y - \hat{f}(x))^2] = \text{Bias}[\hat{f}(x)]^2 + \text{Var}[\hat{f}(x)] + \sigma^2$$

Where:

- y is the true value we are trying to predict.
- $E[\cdot]$ denotes the expected value.
- $\text{Bias}[\hat{f}(x)]^2$ is the squared bias of the model.
- $\text{Var}[\hat{f}(x)]$ is the variance of the model.
- σ^2 is the irreducible error.

Let's break down each of these components:

- **Bias:** Bias is the error caused by using a model that is too simple to capture the true patterns in the data. A high-bias model makes strong assumptions about how the data behaves—often assuming the relationship between inputs and outputs is simple (like linear), even if it's actually complex. This is also known as **underfitting**. For example, trying to fit a linear regression model to data that has a clear non-linear relationship will result in high bias. The bias is the difference between the average prediction of our model and the true value we are trying to predict.

- **Variance:** Variance is the error caused by a model being too sensitive to small changes in the training data. A high-variance model learns not only the underlying pattern but also the noise and randomness in the training set. As a result, it may perform very well on the training data but generalize poorly to new, unseen data. This phenomenon is known as **overfitting**. A complex model, such as a deep decision tree or a neural network with many layers, is prone to high variance. Variance measures how much the predictions for a given point vary across different training sets.
- **Irreducible Error:** This component of the error is due to the inherent noise in the data itself. It represents the lower bound on the expected error of any learning algorithm on the given data. This error cannot be reduced by choosing a better model; it is a characteristic of the data.

The Bias-Variance Tradeoff

The relationship between bias and variance is often inverse: decreasing one tends to increase the other. This is known as the **bias-variance tradeoff**, a central challenge in machine learning.

- **Simple Models (Low Complexity):** Models with low complexity, such as linear regression or shallow decision trees, tend to have high bias and low variance. Their strong assumptions prevent them from capturing the true complexity of the data (high bias), but they are less affected by the specifics of the training data (low variance).
- **Complex Models (High Complexity):** Conversely, models with high complexity, like deep neural networks or very deep decision trees, tend to have low bias but high variance. Their flexibility allows them to fit the training data very closely (low bias), but this also makes them susceptible to capturing the noise in the data, leading to poor generalization (high variance).

The goal is to find a model that strikes an optimal balance between bias and variance, minimizing the total error. This is often visualized as a U-shaped curve where the total error is plotted against model complexity. The sweet spot is at the bottom of the "U," where the sum of squared bias and variance is at its minimum.

Managing Bias and Variance

Understanding the source of a model's error—whether it's high bias or high variance—is key to improving its performance. Here are some common strategies:

To reduce high bias (underfitting):

- **Increase Model Complexity:** Use a more powerful model, such as a polynomial regression instead of a linear one, or a deeper neural network.
- **Add More Features:** Provide the model with more information by engineering new features that might be relevant to the prediction task.

- **Reduce Regularization:** If regularization techniques are being used, decreasing their strength can allow the model to fit the data more closely.

To reduce high variance (overfitting):

- **Increase the Amount of Training Data:** More data can help the model learn the true underlying patterns and be less influenced by noise.
- **Reduce Model Complexity:** Use a simpler model or reduce the number of features.
- **Regularization:** Techniques like L1 and L2 regularization add a penalty term to the loss function to discourage overly complex models.
- **Cross-Validation:** This technique helps in getting a more robust estimate of the model's performance on unseen data and can aid in hyperparameter tuning.
- **Ensemble Methods:** Techniques like Bagging (e.g., Random Forests) and Boosting (e.g., Gradient Boosting Machines) combine multiple models to produce a more robust and accurate prediction, often effectively reducing variance.

In conclusion, the bias-variance decomposition is a powerful theoretical tool that provides deep insights into the behavior of machine learning models. By understanding the tradeoff between bias and variance, practitioners can better diagnose modeling problems and apply appropriate strategies to build more accurate and generalizable predictive models.

Assessing Model Accuracy

Model accuracy, particularly its ability to generalise to unseen data, is crucial. This involves understanding the bias-variance trade-off, which dictates a model's test error.

The Trade-Off

The **bias-variance trade-off** is the inverse relationship between bias and variance when assessing a model's accuracy. It's often expressed in terms of the **Expected Test Mean Squared Error (MSE)**, which can be decomposed as:

$$E[\text{Test MSE}] = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error}$$

- **Irreducible Error:** This is the inherent noise in the data itself that no model, no matter how perfect, can eliminate. It sets a lower bound on the achievable test error.

Here's how the trade-off works:

- **Increasing Model Flexibility/Complexity:**
 - As you make a statistical learning method more flexible (e.g., using a higher-degree polynomial, a deeper decision tree, or less-smoothed splines), its

ability to fit the training data increases.

- This generally **decreases bias**: The model can better capture intricate patterns and reduce systematic errors in its predictions.
- However, this also generally **increases variance**: The model becomes more sensitive to the specific noise and random fluctuations in the training data, making its predictions less stable across different possible training sets.
- **Decreasing Model Flexibility/Complexity:**
 - As you make a statistical learning method less flexible (e.g., using simple linear regression, a shallow decision tree, or highly smoothed splines), its ability to fit the training data is constrained.
 - This generally **increases bias**: The model might be too simplistic to capture the true underlying relationship, leading to consistent systematic errors (underfitting).
 - However, this generally **decreases variance**: The model becomes more stable and less affected by noise in the training data.

The Goal:

The central challenge in statistical learning is to find the **optimal balance** between bias and variance that minimizes the overall **Test MSE**.

- If a model has very **high bias** (underfitting), increasing its flexibility will likely reduce its test MSE by significantly lowering the bias, even if variance increases slightly.
- If a model has very **high variance** (overfitting), decreasing its flexibility (or applying regularization techniques) will reduce its test MSE by significantly lowering the variance, even if bias increases slightly.

The "ideal" model is one that achieves a good balance, having both relatively low bias and low variance, leading to the best generalization performance on unseen data. This U-shaped curve for test MSE as a function of model flexibility is a hallmark of the bias-variance trade-off.

In []: